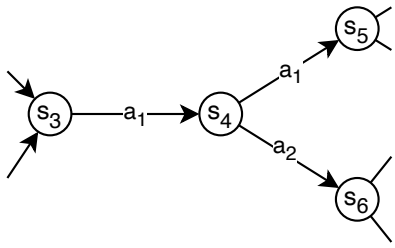
A Clearer Proof of the Policy Gradient Theorem

The policy gradient theorem is a foundational result in reinforcement learning. It turns the derivative of the total reward (hard!) into the derivative of the policy (easy!). Unfortunately, the usual proof uses more algebraic manipulation than you'd like in a foundational result.

Instead, this proof starts with an unusual view of reinforcement learning problems, a view from which you only need a small amount of maths to get to the theorem.

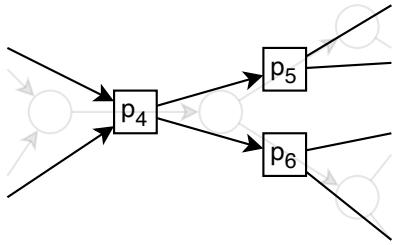
The View

A common way to think about reinforcement learning is that each timestep you find yourself sitting in a state, visualized as a node in a graph. You choose an action - an edge - out of the current state, and the environment takes you from that edge into a new node for the next timestep.



This two-stage transition - node to edge and edge to node - is tricky to work with. It'd be much better if there were only one stage, but you can't ignore the edges entirely because there's valuable information - like rewards - that are defined by which action you take.

What you can do instead is replace each edge with it's own node, representing a *state-action pair*:



Now each timestep has one stage: you're in a node, corresponding to a state-action, and between you and the environment you - somehow - choose a new node for the next timestep. This muddies the boundary between agent's behaviour and environment's behaviour, but for our purposes of proving the policy gradient theorem that's not a problem.

The Notation

In this way of thinking, you can stack the probability of finding yourself in each state-action to get a *position vector* p. This means that p_i is the probability of being in state-action i.

Then to go from one position vector to the next, you simply multiply your current position vector by the transition matrix:

$$p' = p\Pi$$

In this matrix, element Π_{ij} gives the probability of going from state-action i to state-action j. Π merges the behaviour of the environment with the behaviour of the agent, but again that's fine because the policy gradient theorem doesn't distinguish one from the other.

Since transitioning is now simple as multiplication, if you start in a position distribution p_0 then your position after k steps has a nice form:

$$p^{(k)} = p_0\Pi^k$$

More, if r is a column vector of rewards for each state-action then the reward earned in the kth step has a similarly nice form:

$$r^{(k)} = p^{(k)} \cdot r$$

With this notation in hand, the total reward earned by a policy can be written as

$$J = \sum r^{(k)} = \sum p_0\Pi^k r$$

This formula for the total reward can be parsed two ways. The first way is in terms of the *state-action frequency* d:

$$d = \sum p_0\Pi^k \qquad \Longrightarrow \qquad J = d \cdot r$$

This says the total reward is how often you end up in each state-action, multiplied by the reward you get from being there.

The second way to parse the total reward is in terms of the *state-action value* q:

$$q = \sum \Pi^k r \qquad \Longrightarrow \qquad J = p_0 \cdot q$$

This says the total reward is simply the value of the initial state.

To recap,

- p_0 is a row vector giving the initial state-action distribution.
- r is a column vector of the rewards for each state-action.
- Π is the transition matrix, which gives the probability of going from one state-action to another under the policy.
- d is a row vector of the frequencies of visiting each state-action under the policy.
- q is a column vector giving the values - the total future reward - of each state-action under the

policy.
- J is the total reward earned by the policy.

The Simple Proof

The simple way to finish the proof is to act like all the matrices are actually real numbers. Then you can use the geometric formula to write $\sum \Pi^k = \frac{I}{I-\Pi}$, so that

$$d = \frac{p_0}{I-\Pi} \qquad J = p_0 \frac{I}{I-\Pi} r \qquad q = \frac{r}{I-\Pi}$$

This matrix $\frac{I}{I-\Pi}$ has a simple interpretation: element ij is the number of times you'll visit state-action j if you start in state-action i.

Once you've got these three definitions, you can use high-school calculus to work out that the gradient of the total reward is

$$J' = p_0 \left(\frac{I}{I-\Pi} \right)' r = p_0 \frac{\Pi'}{(I-\Pi)^2} r = d\Pi' q$$

That's the policy gradient theorem: $J' = d\Pi' q$. The left-hand side is hard, the right-hand side is easy. QED.

Kinda.

This approach 'works' in that it gives the right answer using rules you're familiar with, but it involves more than one illegal operation. For one, $I - \Pi$ can't be inverted. To do things properly requires a bit more indirection.

The Correct Proof

Although q and d are easiest to intuit as sums, multiplying their definitions through by Π gives a neat defining identity for each one:

$$d = d\Pi + p_0 \qquad q = \Pi q + r$$

or - gathering the terms -

$$p_0 = d(I - \Pi) \qquad r = (I - \Pi) q$$

They can also be turned into neat identities for the gradient:

$$d'(I - \Pi) = d\Pi' \qquad (I - \Pi) q' = \Pi' q$$

Given these identities, the policy gradient theorem falls out in two symmetric ways. The gradient of the total reward is

$$J' = d' \cdot r = d'(I - \Pi)q = d\Pi' q \qquad J' = p_0 \cdot q' = d(I - \Pi)q' = d\Pi' q$$

Once more, $J' = d\Pi' q$. QED.

Notes

An especially nice thing about this notation is that $J' = d\Pi' q$ has an obvious interpretation. J' will be positive when Π' increases the likelihood of transitions that go from oft-visited state-actions to high-value state-actions. Just the behaviour you want!

You can find the usual proof of the policy-gradient theorem on §13.2/p269 of Sutton & Barto. The usual proof is more compact, but requires an infinite telescope of the gradient that confuses things. You can find a version with more commentary here, or an alternative approach based on expectations here.

Thanks to Adam Gleave and Jacob Hilton for comments and suggestions, and to Jerry Lin for corrections.

Citation

```
@article{jones2020CPPGT,
  title   = "A Clearer Proof of the Policy Gradient Theorem",
  author  = "Andy L. Jones",
  journal = "andyljones.com",
  year    = "2020",
  url     = "https://andyljones.com/posts/policy-gradient.html"
}
```

2020/05/25

icons by dave gandy, theme by #6d2e98

Policy Gradient Algorithms

Date: April 8, 2018 | Estimated Reading Time: 52 min | Author: Lilian Weng

▶ Table of Contents

[Updated on 2018-06-30: add two new policy gradient methods, SAC and D4PG.]
[Updated on 2018-09-30: add a new policy gradient method, TD3.]
[Updated on 2019-02-09: add SAC with automatically adjusted temperature].
[Updated on 2019-06-26: Thanks to Chanseok, we have a version of this post in Korean].
[Updated on 2019-09-12: add a new policy gradient method SVPG.]
[Updated on 2019-12-22: add a new policy gradient method IMPALA.]
[Updated on 2020-10-15: add a new policy gradient method PPG & some new discussion in PPO.]
[Updated on 2021-09-19: Thanks to Wenhao & 爱吃猫的鱼, we have this post in Chinese1 & Chinese2].

What is Policy Gradient

Policy gradient is an approach to solve reinforcement learning problems. If you haven't looked into the field of reinforcement learning, please first read the section "A (Long) Peek into Reinforcement Learning » Key Concepts" for the problem definition and key concepts.

Notations

Here is a list of notations to help you read through equations in the post easily.

Symbol	Meaning
$s \in \mathcal{S}$	States.
$a \in \mathcal{A}$	Actions.
$r \in \mathcal{R}$	Rewards.
S_t, A_t, R_t	State, action, and reward at time step t of one trajectory. I may occasionally use s_t, a_t, r_t as well.
γ	Discount factor; penalty to uncertainty of future rewards; $0 < \gamma \leq 1$.
G_t	Return; or discounted future reward; $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$.
$P(s', r \vert s, a)$	Transition probability of getting to the next state s' from the current state s with action a and reward r.
$\pi(a \vert s)$	Stochastic policy (agent behavior strategy); $\pi_\theta(.)$ is a policy parameterized by θ.

$\mu(s)$	Deterministic policy; we can also label this as $\pi(s)$, but using a different letter gives better distinction so that we can easily tell when the policy is stochastic or deterministic without further explanation. Either π or μ is what a reinforcement learning algorithm aims to learn.
$V(s)$	State-value function measures the expected return of state s; $V_w(.)$ is a value function parameterized by w.
$V^\pi(s)$	The value of state s when we follow a policy π; $V^\pi(s) = \mathbb{E}_{a\sim\pi}[G_t \vert S_t = s]$.
$Q(s, a)$	Action-value function is similar to $V(s)$, but it assesses the expected return of a pair of state and action (s, a); $Q_w(.)$ is a action value function parameterized by w.
$Q^\pi(s, a)$	Similar to $V^\pi(.)$, the value of (state, action) pair when we follow a policy π; $Q^\pi(s, a) = \mathbb{E}_{a\sim\pi}[G_t \vert S_t = s, A_t = a]$.
$A(s, a)$	Advantage function, $A(s, a) = Q(s, a) - V(s)$; it can be considered as another version of Q-value with lower variance by taking the state-value off as the baseline.

Policy Gradient

The goal of reinforcement learning is to find an optimal behavior strategy for the agent to obtain optimal rewards. The **policy gradient** methods target at modeling and optimizing the policy directly. The policy is usually modeled with a parameterized function respect to θ, $\pi_\theta(a \vert s)$. The value of the reward (objective) function depends on this policy and then various algorithms can be applied to optimize θ for the best reward.

The reward function is defined as:

$$J(\theta) = \sum_{s\in\mathcal{S}} d^\pi(s) V^\pi(s) = \sum_{s\in\mathcal{S}} d^\pi(s) \sum_{a\in\mathcal{A}} \pi_\theta(a \vert s) Q^\pi(s, a)$$

where $d^\pi(s)$ is the stationary distribution of Markov chain for π_θ (on-policy state distribution under π). For simplicity, the parameter θ would be omitted for the policy π_θ when the policy is present in the subscript of other functions; for example, d^π and Q^π should be d^{π_θ} and Q^{π_θ} if written in full.

Imagine that you can travel along the Markov chain's states forever, and eventually, as the time progresses, the probability of you ending up with one state becomes unchanged — this is the stationary probability for π_θ. $d^\pi(s) = \lim_{t\to\infty} P(s_t = s \vert s_0, \pi_\theta)$ is the probability that $s_t = s$ when starting from s_0 and following policy π_θ for t steps. Actually, the existence of the stationary distribution of Markov chain is one main reason for why PageRank algorithm works. If you want to read more, check this.

It is natural to expect policy-based methods are more useful in the continuous space. Because there is an infinite number of actions and (or) states to estimate the values for and hence value-based approaches are way too expensive computationally in the continuous space. For example, in generalized policy iteration, the policy improvement step $\arg\max_{a\in\mathcal{A}} Q^\pi(s, a)$ requires a full scan of the action space, suffering from the curse of dimensionality.

Using *gradient ascent*, we can move θ toward the direction suggested by the gradient $\nabla_\theta J(\theta)$ to

find the best θ for π_θ that produces the highest return.

Policy Gradient Theorem

Computing the gradient $\nabla_\theta J(\theta)$ is tricky because it depends on both the action selection (directly determined by π_θ) and the stationary distribution of states following the target selection behavior (indirectly determined by π_θ). Given that the environment is generally unknown, it is difficult to estimate the effect on the state distribution by a policy update.

Luckily, the **policy gradient theorem** comes to save the world! Woohoo! It provides a nice reformation of the derivative of the objective function to not involve the derivative of the state distribution $d^\pi(.)$ and simplify the gradient computation $\nabla_\theta J(\theta)$ a lot.

$$\begin{aligned}\nabla_\theta J(\theta) &= \nabla_\theta \sum_{s\in\mathcal{S}} d^\pi(s) \sum_{a\in\mathcal{A}} Q^\pi(s,a)\pi_\theta(a|s) \\ &\propto \sum_{s\in\mathcal{S}} d^\pi(s) \sum_{a\in\mathcal{A}} Q^\pi(s,a)\nabla_\theta\pi_\theta(a|s)\end{aligned}$$

Proof of Policy Gradient Theorem

This session is pretty dense, as it is the time for us to go through the proof (Sutton & Barto, 2017; Sec. 13.1) and figure out why the policy gradient theorem is correct.

We first start with the derivative of the state value function:

$$\begin{aligned}
&\nabla_\theta V^\pi(s) \\
&= \nabla_\theta\Big(\sum_{a\in\mathcal{A}} \pi_\theta(a|s)Q^\pi(s,a)\Big) \\
&= \sum_{a\in\mathcal{A}} \Big(\nabla_\theta\pi_\theta(a|s)Q^\pi(s,a) + \pi_\theta(a|s)\nabla_\theta Q^\pi(s,a)\Big) && \text{; Derivative product rule.} \\
&= \sum_{a\in\mathcal{A}} \Big(\nabla_\theta\pi_\theta(a|s)Q^\pi(s,a) + \pi_\theta(a|s)\nabla_\theta \sum_{s',r} P(s',r|s,a)(r+V^\pi(s'))\Big) && \text{; Extend } Q^\pi \text{ with future state value.} \\
&= \sum_{a\in\mathcal{A}} \Big(\nabla_\theta\pi_\theta(a|s)Q^\pi(s,a) + \pi_\theta(a|s)\sum_{s',r} P(s',r|s,a)\nabla_\theta V^\pi(s')\Big) && P(s',r|s,a) \text{ or } r \text{ is not a func of } \theta \\
&= \sum_{a\in\mathcal{A}} \Big(\nabla_\theta\pi_\theta(a|s)Q^\pi(s,a) + \pi_\theta(a|s)\sum_{s'} P(s'|s,a)\nabla_\theta V^\pi(s')\Big) && \text{; Because } P(s'|s,a)=\textstyle\sum_r P(s',r|s,a)
\end{aligned}$$

Now we have:

$$\nabla_\theta V^\pi(s) = \sum_{a\in\mathcal{A}} \Big(\nabla_\theta\pi_\theta(a|s)Q^\pi(s,a) + \pi_\theta(a|s)\sum_{s'} P(s'|s,a)\nabla_\theta V^\pi(s')\Big)$$

This equation has a nice recursive form (see the red parts!) and the future state value function $V^\pi(s')$ can be repeated unrolled by following the same equation.

Let's consider the following visitation sequence and label the probability of transitioning from state s to state x with policy π_θ after k step as $\rho^\pi(s\to x,k)$.

$$s \xrightarrow{a\sim\pi_\theta(.|s)} s' \xrightarrow{a\sim\pi_\theta(.|s')} s'' \xrightarrow{a\sim\pi_\theta(.|s'')} \dots$$

- When k = 0: $\rho^\pi(s \to s, k=0) = 1$.
- When k = 1, we scan through all possible actions and sum up the transition probabilities to the target state: $\rho^\pi(s \to s', k=1) = \sum_a \pi_\theta(a|s)P(s'|s,a)$.
- Imagine that the goal is to go from state s to x after k+1 steps while following policy π_θ. We can first travel from s to a middle point s′ (any state can be a middle point, $s' \in \mathcal{S}$) after k steps and then go to the final state x during the last step. In this way, we are able to update the visitation probability recursively: $\rho^\pi(s \to x, k+1) = \sum_{s'} \rho^\pi(s \to s', k)\rho^\pi(s' \to x, 1)$.

Then we go back to unroll the recursive representation of $\nabla_\theta V^\pi(s)$! Let $\phi(s) = \sum_{a\in\mathcal{A}} \nabla_\theta \pi_\theta(a|s) Q^\pi(s,a)$ to simplify the maths. If we keep on extending $\nabla_\theta V^\pi(.)$ infinitely, it is easy to find out that we can transition from the starting state s to any state after any number of steps in this unrolling process and by summing up all the visitation probabilities, we get $\nabla_\theta V^\pi(s)$!

$$\begin{aligned}
&\nabla_\theta V^\pi(s) \\
=&\phi(s) + \sum_a \pi_\theta(a|s) \sum_{s'} P(s'|s,a) \nabla_\theta V^\pi(s') \\
=&\phi(s) + \sum_{s'} \sum_a \pi_\theta(a|s) P(s'|s,a) \nabla_\theta V^\pi(s') \\
=&\phi(s) + \sum_{s'} \rho^\pi(s \to s', 1) \nabla_\theta V^\pi(s') \\
=&\phi(s) + \sum_{s'} \rho^\pi(s \to s', 1) \sum_{a\in\mathcal{A}} \Big(\nabla_\theta \pi_\theta(a|s') Q^\pi(s',a) + \pi_\theta(a|s') \sum_{s'} P(s''|s',a) \nabla_\theta V^\pi(s'') \Big) \\
=&\phi(s) + \sum_{s'} \rho^\pi(s \to s', 1) [\phi(s') + \sum_{s''} \rho^\pi(s' \to s'', 1) \nabla_\theta V^\pi(s'')] \\
=&\phi(s) + \sum_{s'} \rho^\pi(s \to s', 1) \phi(s') + \sum_{s''} \rho^\pi(s \to s'', 2) \nabla_\theta V^\pi(s'') \text{ ; Consider } s' \text{ as the middle point for } s \to s'' \\
=&\phi(s) + \sum_{s'} \rho^\pi(s \to s', 1) \phi(s') + \sum_{s''} \rho^\pi(s \to s'', 2) \phi(s'') + \sum_{s'''} \rho^\pi(s \to s''', 3) \nabla_\theta V^\pi(s''') \\
=&\dots \text{; Repeatedly unrolling the part of } \nabla_\theta V^\pi(.) \\
=&\sum_{x\in\mathcal{S}} \sum_{k=0}^{\infty} \rho^\pi(s \to x, k) \phi(x)
\end{aligned}$$

The nice rewriting above allows us to exclude the derivative of Q-value function, $\nabla_\theta Q^\pi(s,a)$. By plugging it into the objective function $J(\theta)$, we are getting the following:

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta V^\pi(s_0) && \text{; Starting from a random state } s_0 \\
&= \sum_s \sum_{k=0}^{\infty} \rho^\pi(s_0 \to s, k)\phi(s) && \text{; Let } \eta(s)=\textstyle\sum_{k=0}^{\infty} \rho^\pi(s_0 \to s, k) \\
&= \sum_s \eta(s)\phi(s) \\
&= \Big(\sum_s \eta(s)\Big) \sum_s \frac{\eta(s)}{\sum_s \eta(s)}\phi(s) && \text{; Normalize } \eta(s), s\in\mathcal{S} \text{ to be a probability distribution.} \\
&\propto \sum_s \frac{\eta(s)}{\sum_s \eta(s)}\phi(s) && \textstyle\sum_s \eta(s) \text{ is a constant} \\
&= \sum_s d^\pi(s) \sum_a \nabla_\theta \pi_\theta(a|s) Q^\pi(s,a) && d^\pi(s)=\frac{\eta(s)}{\sum_s \eta(s)} \text{ is stationary distribution.}
\end{aligned}
$$

In the episodic case, the constant of proportionality ($\sum_s \eta(s)$) is the average length of an episode; in the continuing case, it is 1 (Sutton & Barto, 2017; Sec. 13.2). The gradient can be further written as:

$$
\begin{aligned}
\nabla_\theta J(\theta) &\propto \sum_{s\in\mathcal{S}} d^\pi(s) \sum_{a\in\mathcal{A}} Q^\pi(s,a) \nabla_\theta \pi_\theta(a|s) \\
&= \sum_{s\in\mathcal{S}} d^\pi(s) \sum_{a\in\mathcal{A}} \pi_\theta(a|s) Q^\pi(s,a) \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)} \\
&= \mathbb{E}_\pi[Q^\pi(s,a) \nabla_\theta \ln \pi_\theta(a|s)] && \text{; Because } (\ln x)'=1/x
\end{aligned}
$$

Where $\mathbb{E}_\pi$ refers to $\mathbb{E}_{s\sim d_\pi, a\sim\pi_\theta}$ when both state and action distributions follow the policy π_θ (on policy).

The policy gradient theorem lays the theoretical foundation for various policy gradient algorithms. This vanilla policy gradient update has no bias but high variance. Many following algorithms were proposed to reduce the variance while keeping the bias unchanged.

$$
\nabla_\theta J(\theta) = \mathbb{E}_\pi[Q^\pi(s,a) \nabla_\theta \ln \pi_\theta(a|s)]
$$

Here is a nice summary of a general form of policy gradient methods borrowed from the GAE (general advantage estimation) paper (Schulman et al., 2016) and this post thoroughly discussed several components in GAE , highly recommended.